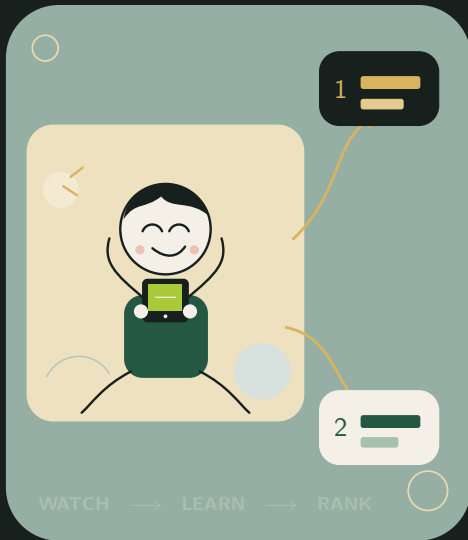


Recommendation Algorithms

How a feed turns small signals into a ranked guess.

TEAM MEMBERS

2305052 / 2305055 / 2305033



Your next favorite.

A story about how a feed
learns what to show you.

Your next favorite.

A story about how a feed learns what to show you.

Meet Bob.

Saturday morning. One funny video.



Sunday looks familiar.

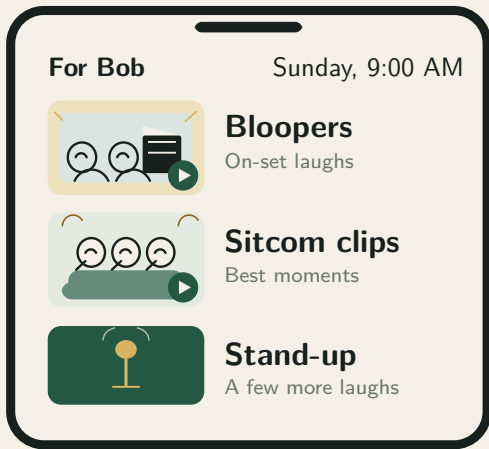
One video yesterday.



Sunday looks familiar.

One video yesterday.

A feed full of comedy.

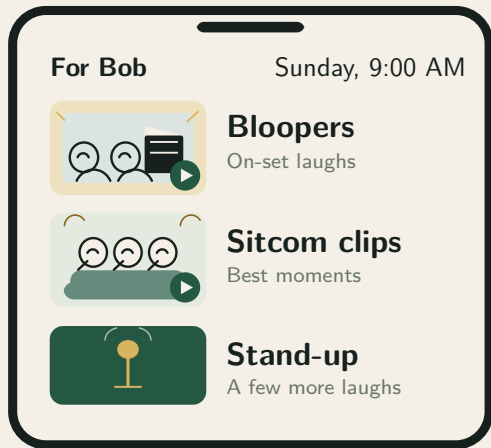


Sunday looks familiar.

One video yesterday.
A feed full of comedy.

Coincidence?

Something is happening
behind the screen.



A ranked guess.

The system uses evidence to decide what Bob might enjoy next.

Observe

Watches,
ratings, skips

A ranked guess.

The system uses evidence to decide what Bob might enjoy next.

Observe

Watches,
ratings, skips

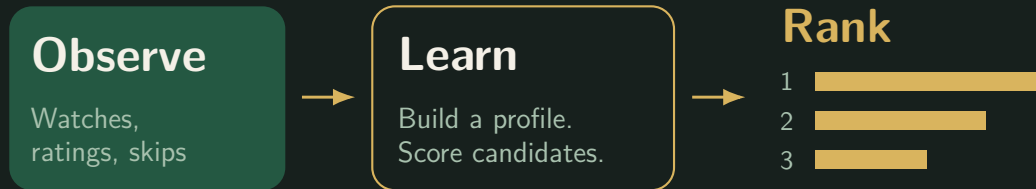


Learn

Build a profile.
Score candidates.

A ranked guess.

The system uses evidence to decide what Bob might enjoy next.



That is the job of a **recommendation system**.

Bob tells us in two ways.

EXPLICIT FEEDBACK

What he says.



Rate. Like. Dislike.

A direct expression of preference.

Bob tells us in two ways.

EXPLICIT FEEDBACK

What he says.



Rate. Like. Dislike.

A direct expression of preference.

IMPLICIT FEEDBACK

What he does.



Watch. Replay. Skip.

Behavior that may suggest interest.

Bob tells us in two ways.

EXPLICIT FEEDBACK

What he says.



Rate. Like. Dislike.

A direct expression of preference.

IMPLICIT FEEDBACK

What he does.



Watch. Replay. Skip.

Behavior that may suggest interest.

A long watch is a clue, not proof that Bob liked it.

A few ratings. Many unknowns.

Rows = people. Columns = videos. Entries = ratings.

	Bloopers	Sitcom 1	Action	Travel
Bob →	5			
Asha				
Leo				

Bob rated
bloopers **5**.

A few ratings. Many unknowns.

Rows = people. Columns = videos. Entries = ratings.

	Bloopers	Sitcom 1	Action	Travel
Bob	5	4	?	?
Asha	4	?	2	?
Leo	?	?	5	?

Most ratings
are missing.

A few ratings. Many unknowns.

Rows = people. Columns = videos. Entries = ratings.

	Bloopers	Sitcom 1	Action	Travel
Bob	5	4	?	?
Asha	4	?	2	?
Leo	?	?	5	?

Most ratings
are missing.

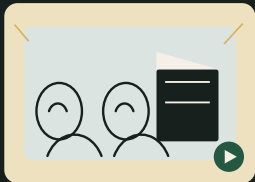
Unknown
 $\neq$ disliked.

The challenge: score a video Bob has never rated.

Describe videos. Learn Bob's taste.

Recommend videos whose features resemble what Bob already enjoys.

VIDEOS BOB ENJOYS

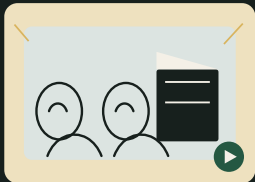


Movie bloopers

Describe videos. Learn Bob's taste.

Recommend videos whose features resemble what Bob already enjoys.

VIDEOS BOB ENJOYS



Movie bloopers



ITEM FEATURES

Comedy

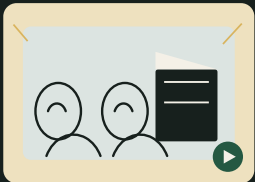
Genre, tone, topic,
cast, duration...

Describes the video.

Describe videos. Learn Bob's taste.

Recommend videos whose features resemble what Bob already enjoys.

VIDEOS BOB ENJOYS



Movie bloopers

ITEM FEATURES

Comedy

Genre, tone, topic,
cast, duration...

Describes the video.

BOB'S CONTENT PROFILE

$$\mathbf{p} = (3, 1)$$

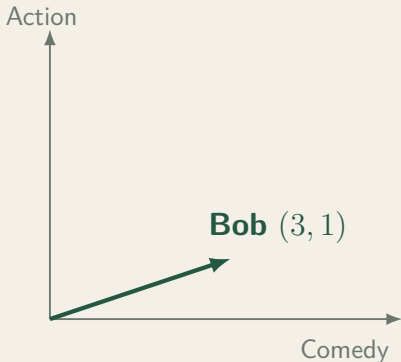
Comedy: 3 Action: 1

From his feedback
and video features.

A profile of **features**, built from a history of **feedback**.

The closer direction wins.

Compare Bob's profile with candidate video features using cosine similarity.



CANDIDATE VIDEO FEATURES: (COMEDY, ACTION)



A / Sitcom 2

$a = (4, 0)$



B / Action trailer

$b = (1, 4)$

The closer direction wins.

Compare Bob's profile with candidate video features using cosine similarity.

Action

CANDIDATE VIDEO FEATURES: (COMEDY, ACTION)



A / Sitcom 2

$a = (4, 0)$

0.95



B / Action trailer

$b = (1, 4)$

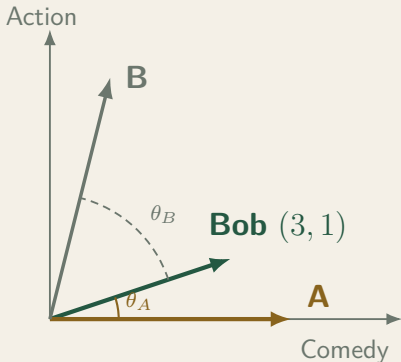
Bob (3, 1)

θ_A

A
Comedy

The closer direction wins.

Compare Bob's profile with candidate video features using cosine similarity.



CANDIDATE VIDEO FEATURES: (COMEDY, ACTION)



A / Sitcom 2

$a = (4, 0)$

0.95



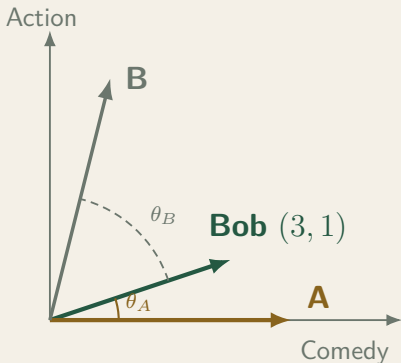
B / Action trailer

$b = (1, 4)$

0.54

The closer direction wins.

Compare Bob's profile with candidate video features using cosine similarity.



CANDIDATE VIDEO FEATURES: (COMEDY, ACTION)



A / Sitcom 2

$\mathbf{a} = (4, 0)$

0.95



B / Action trailer

$\mathbf{b} = (1, 4)$

0.54

$$\text{sim}(\mathbf{p}, \mathbf{v}) = \frac{\mathbf{p} \cdot \mathbf{v}}{\|\mathbf{p}\| \|\mathbf{v}\|}$$

$0.95 > 0.54$

Recommend A.

Can Bob discover something different?

Familiar features can mean more of the same.

| What if people with Bob's taste love a video with completely different tags?

Can Bob discover something different?

Familiar features can mean more of the same.

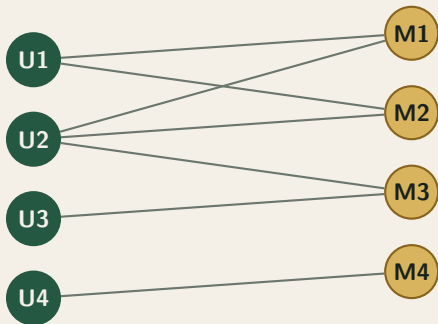
| What if people with Bob's taste love a video with completely different tags?

Collaborative filtering

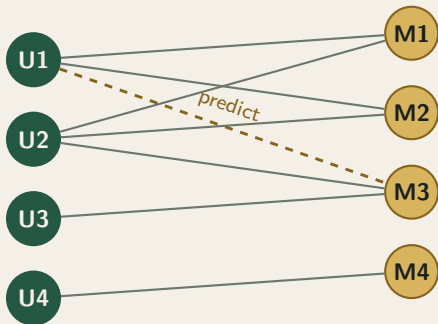


My teammate takes the story from here.

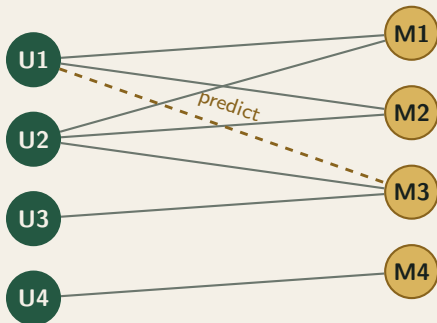
Collaborative filtering



Collaborative filtering



Collaborative filtering



Collaborative signal

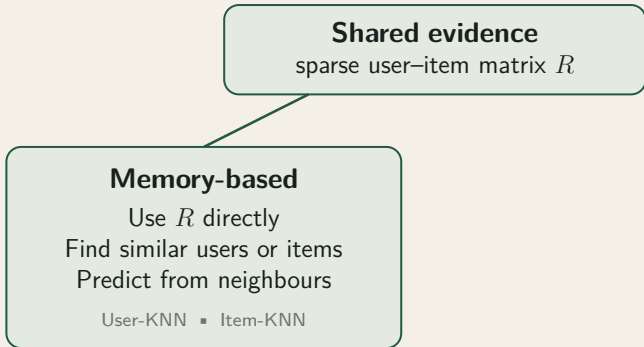
If users with similar histories liked M_3 , estimate that U_1 may also like M_3 .

Collaborative filtering has two main families

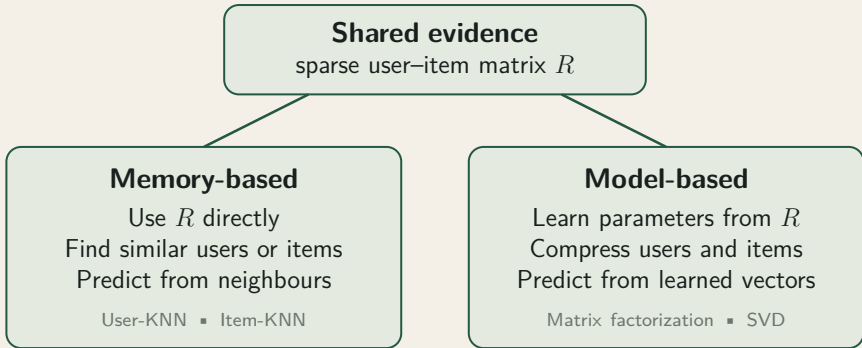
Shared evidence

sparse user–item matrix R

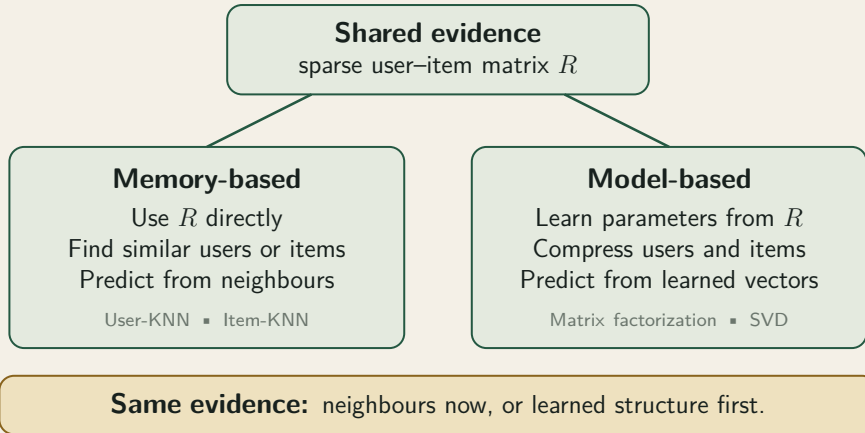
Collaborative filtering has two main families



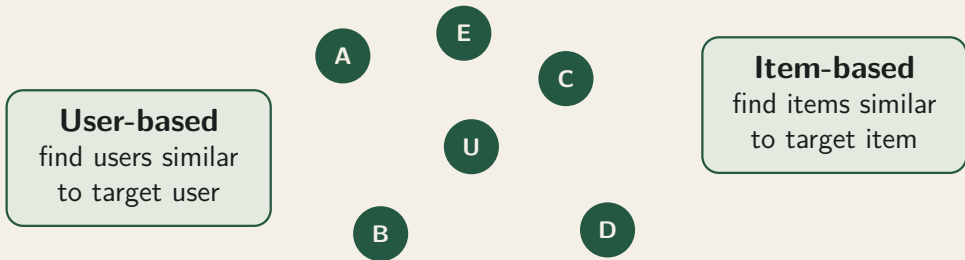
Collaborative filtering has two main families



Collaborative filtering has two main families

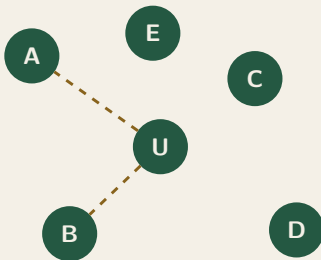


Memory-based methods



Memory-based methods

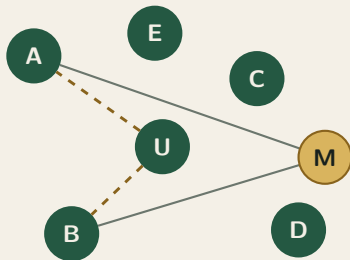
User-based
find users similar
to target user



Item-based
find items similar
to target item

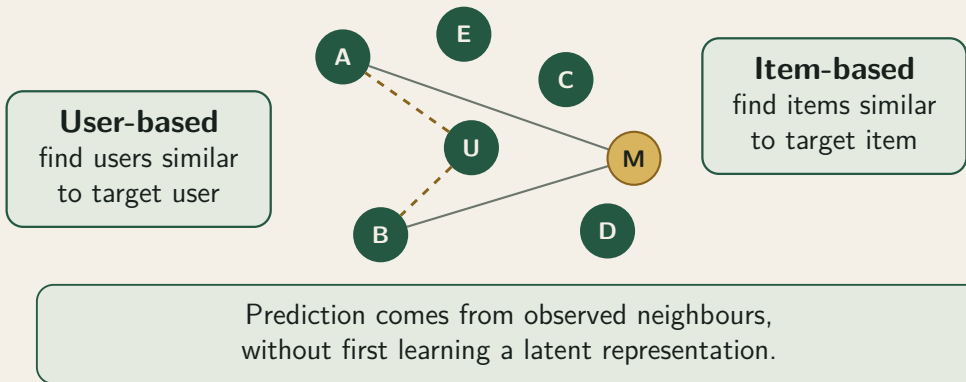
Memory-based methods

User-based
find users similar
to target user



Item-based
find items similar
to target item

Memory-based methods



Why memory-based methods struggle at scale

Sparsity

two users may
overlap on
very few items

Why memory-based methods struggle at scale

Sparsity

two users may
overlap on
very few items

Scalability

similarity over
millions of
users/items
is expensive

Why memory-based methods struggle at scale

Sparsity

two users may
overlap on
very few items

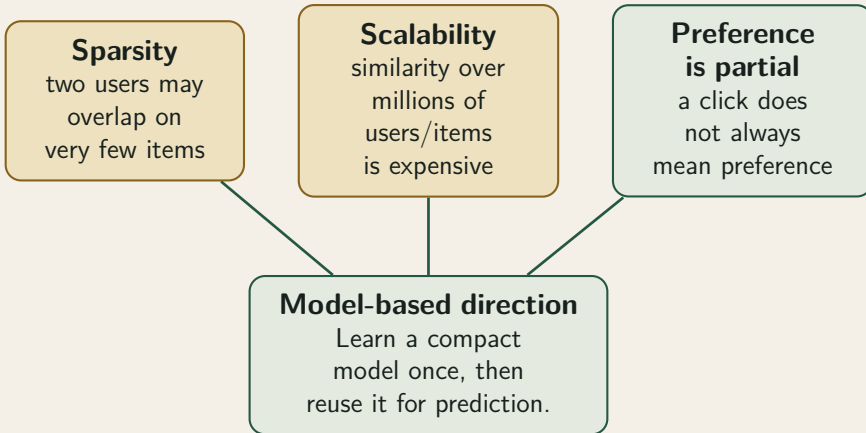
Scalability

similarity over
millions of
users/items
is expensive

Preference is partial

a click does
not always
mean preference

Why memory-based methods struggle at scale



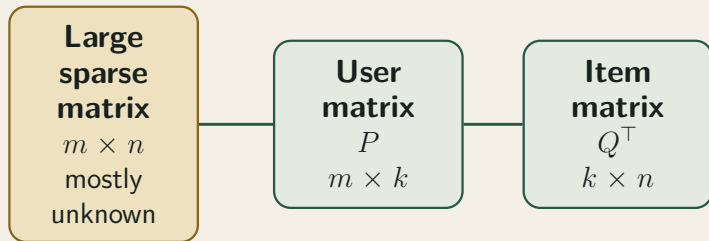
Model-based latent features

**Large
sparse
matrix**

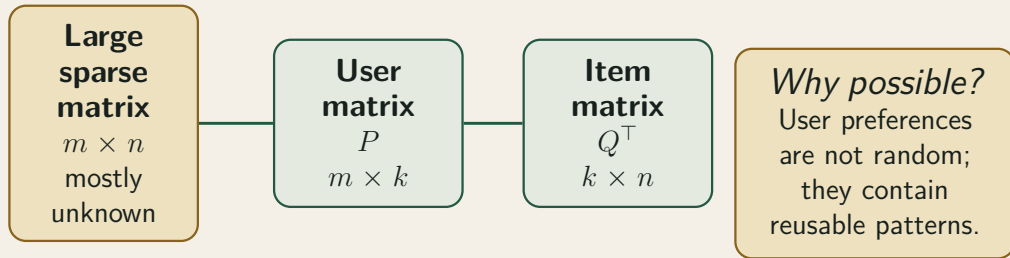
$m \times n$

mostly
unknown

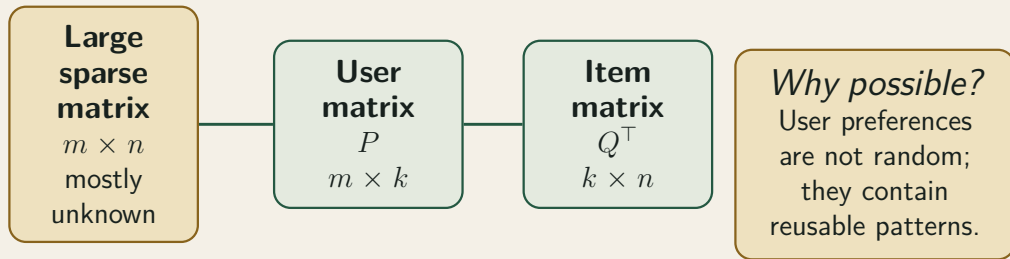
Model-based latent features



Model-based latent features



Model-based latent features



The model may discover axes such as “mainstream vs niche” or “light comedy vs dark drama” without human labels.

How do these methods work in practice?

We have seen the methodology

Collaborative evidence can be used
directly or compressed into latent factors.

How do these methods work in practice?

We have seen the methodology

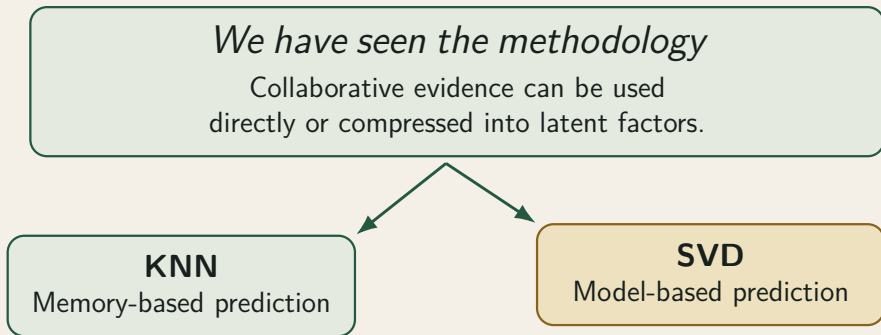
Collaborative evidence can be used
directly or compressed into latent factors.



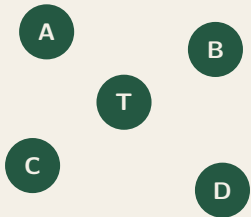
KNN

Memory-based prediction

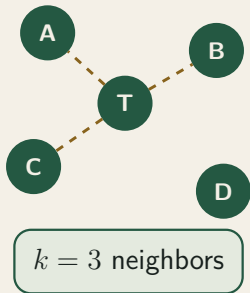
How do these methods work in practice?



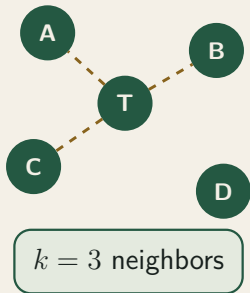
KNN predicts from nearest neighbours



KNN predicts from nearest neighbours



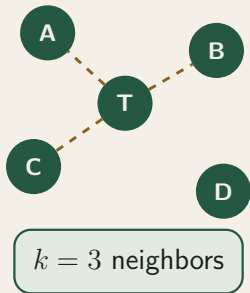
KNN predicts from nearest neighbours



Weighted prediction

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_{v \in N(u)} s(u, v)(r_{vi} - \bar{r}_v)}{\sum_{v \in N(u)} |s(u, v)|}$$

KNN predicts from nearest neighbours



Weighted prediction

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_{v \in N(u)} s(u, v)(r_{vi} - \bar{r}_v)}{\sum_{v \in N(u)} |s(u, v)|}$$

The neighborhood provides local evidence, usually measured by cosine or Pearson similarity.

SVD learns latent factors

Observed

R

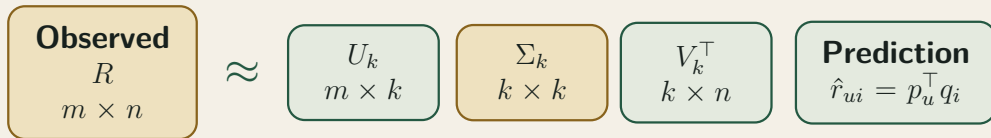
$m \times n$

SVD learns latent factors

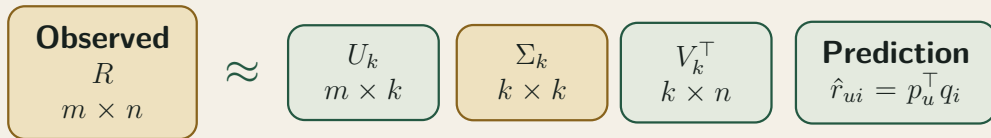
The diagram illustrates the SVD decomposition of an observed matrix R into three latent factor matrices. On the left, a yellow box labeled "Observed" contains the matrix R with dimensions $m \times n$. To its right is a teal approximation symbol $\approx$. Further right are three boxes: a teal box for U_k with dimensions $m \times k$, a yellow box for Σ_k with dimensions $k \times k$, and a teal box for $V_k^\top$ with dimensions $k \times n$.

$$\begin{array}{|c|} \hline \text{Observed} \\ \hline R \\ m \times n \\ \hline \end{array} \approx \begin{array}{|c|} \hline U_k \\ m \times k \\ \hline \end{array} \begin{array}{|c|} \hline \Sigma_k \\ k \times k \\ \hline \end{array} \begin{array}{|c|} \hline V_k^\top \\ k \times n \\ \hline \end{array}$$

SVD learns latent factors



SVD learns latent factors



$$\min_{P,Q} \sum_{(u,i) \in \Omega} (r_{ui} - p_u^\top q_i)^2 + \lambda(\|p_u\|^2 + \|q_i\|^2)$$

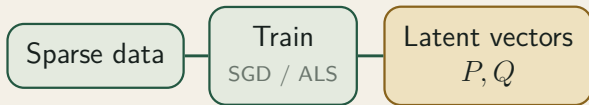
End-to-end recommender workflow

Sparse data

End-to-end recommender workflow



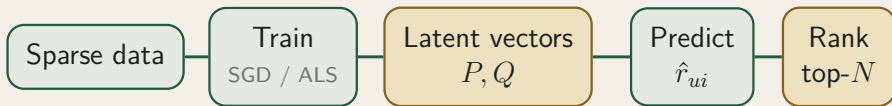
End-to-end recommender workflow



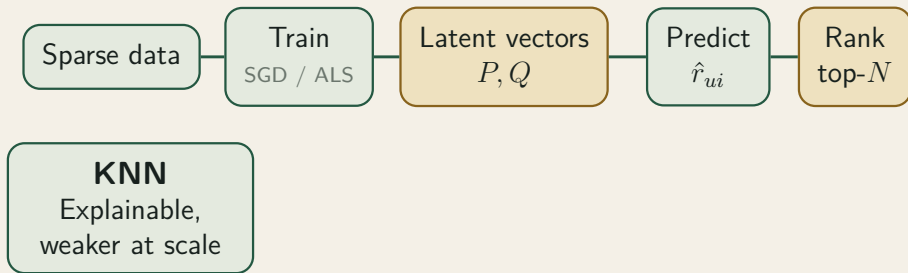
End-to-end recommender workflow



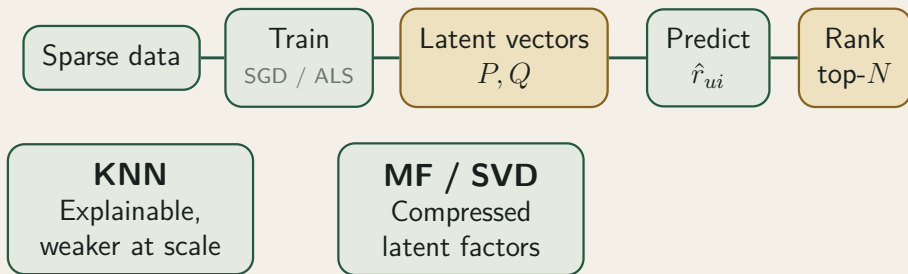
End-to-end recommender workflow



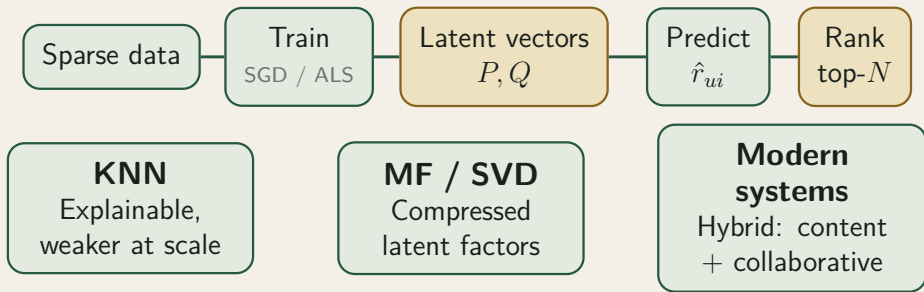
End-to-end recommender workflow



End-to-end recommender workflow



End-to-end recommender workflow



Takeaways

Sparse matrix

Observed
only partially.

Takeaways

Sparse matrix

Observed
only partially.

Two families

Content +
collaborative.

Takeaways

Sparse matrix

Observed
only partially.

Two families

Content +
collaborative.

Scaling idea

Compress into
latent factors.

Takeaways

Sparse matrix

Observed
only partially.

Two families

Content +
collaborative.

Scaling idea

Compress into
latent factors.

Final output

Ranked
personalized
choices.

Takeaways

Sparse matrix

Observed
only partially.

Two families

Content +
collaborative.

Scaling idea

Compress into
latent factors.

Final output

Ranked
personalized
choices.

**The system does not know exactly what we will like
it estimates from patterns shared across users, items and
context.**